

**TME 2 – 3 : Support de la conteneurisation sous Linux et
divers utilitaires**

HUANG Loïc 21203938
HARRAOUI Ines 21204796

I. Inspection du code source : Vue d'ensemble de la conception du noyau pour prendre en charge la conteneurisation

1. Trouvez le fichier nsproxy.h dans votre système (système de la VM CRV4)

1.1. Comparez la liste des espaces de noms (namespaces) mentionnés avec celui listé dans le manuel (man namespaces).

La liste des espaces de noms (namespaces) contenu dans le fichier sont pareil que les mêmes namespaces que celui listé dans le manuel (man namespaces). Cependant, il nous manque le namespace User dans le fichier.

1.2. Pourquoi la structure pid_namespace s'appelle-t-elle pid_ns_for_children ?

La structure pid_namespace s'appelle-t-elle pid_ns_for_children parce qu'elle permet d'indiquer aux processus fils du processus courant, quels namespaces ils doivent être lorsqu'ils seront créés.

2. Site : <https://www.kernel.org/>

Dans le fichier fork.c, trouvez l'implémentation de l'appel système unshare().

2.1. Quels sont les arguments d'entrée de l'appel système unshare() ? Par exemple, si nous voulons annuler le partage de la pile réseau pour le processus appelant, autrement dit dupliquer la pile réseau pour le processus appelant, quelle doit être la valeur numérique de l'argument d'entrée ?

L'argument de la fonction d'appel système unshare() est une valeur en unsigned long nommée unshare_flags. Pour pouvoir annuler le partage de la pile réseau pour le processus appelant, il faudrait utiliser la valeur de la variable clone_newnet dans la fonction unshare().

```
2608 | SYSCALL_DEFINE1(unshare, unsigned long, unshare_flags)
2609 | {
2610 |     return ksys_unshare(unshare_flags);
2611 | }
```

Image 1 : Fonction d'appel système unshare()

2.2. Dans le même appel unshare(), trouvez l'appel de fonction qui permet (et qui déclenche) la création d'une pile réseau distincte. Quelle est la variable qui pointera vers le nouvel espace de noms réseau ?

La fonction qui permet la création d'une pile réseau distincte est `unshare_nsproxy_namespaces`. La variable qui pointera vers le nouvel espace de noms réseau est `new_ns_proxy`.

2.3. Enfin, trouvez les deux lignes qui changent l'espace de noms réseau vers le nouvel espace de noms créé.

Ces deux lignes permettent de changer l'espace de noms réseau vers le nouvel espace de noms créé.

```
2560         if (new_nsproxy)
2561             switch_task_namespaces(current, new_nsproxy);
```

Image 2 : Appel de la fonction `switch_task_namespaces`

3. Fichier `net_namespace.h`

3.1. Trouvez la ligne qui indique la liste des espaces de noms réseau maintenu par le noyau.

La ligne 63 du fichier indique la liste des espaces de noms réseau maintenus par le noyau.

```
63 struct list_head list; /* list of network namespaces */
```

Images 3 : Variable `list`

3.2. Est-ce que les structures de données IPv4 sont-elles différentes d'un espace de noms à un autre ? Comment l'avez-vous vérifié ? Bien Justifiez.

Les structures de données IPv4 diffèrent d'un namespace à l'autre car à chaque fois qu'un namespace est créé, on initialise une nouvelle instance de la variable `ipv4` qui sera indépendante des autres namespaces.

3.3. Quelle est la fonction appelée pour ajouter des opérations d'initialisation et de terminaison (sortie/nettoyage) d'un module d'espace de noms réseau ?

La fonction `register_pernet_subsys()` permet de rajouter des opérations d'initialisation et de terminaison d'un module d'espace de noms réseau.

4. Fichier `icmp.c`

4.1. Est-ce que la fonction identifiée précédemment (question 3.3/) est appelée dans ce fichier ?

La fonction `register_pernet_subsys()` est bien présente dans le fichier, il est utilisée dans plusieurs fonctions / structures du fichier.

4.2. Quel est le nom de la fonction d'initialisation du module ICMP ?
Quelle est la fonction de sortie ?

Module ICMP :

- Fonction d'initialisation : `icmpv6_init(void)`
- Fonction de sorties : `icmpv6_cleanup(void)`

4.3. Pourquoi la plupart des variables du fichier `icmp.h` sont-elles précédées par `net->` ?

La raison pour laquelle la plupart des variables du fichier `icmp.h` sont précédées par `net->` est qu'on doit préciser dans quel namespace on doit utiliser. Si on n'avait pas eu de `net->` on aurait pas pu savoir où le module ICMP serait utilisé.

5. Dans `sys-utils`, fichier `unshare.c`

5.1. Trouvez la ligne où l'appel système `unshare()` est utilisé pour modifier les espaces de noms.

Dans le code, la seule ligne utilisant l'appel système `unshare()` se situe sur la ligne 1112, donc il est probablement utilisé pour modifier les espaces de noms.

```
1112         if (-1 == unshare(unshare_flags))
1113             err(EXIT_FAILURE, _("unshare failed"));
```

Image 4 : Utilisation de la fonction `unshare()`

5.2. Quel est le nom du drapeau `unshare_flag` qui crée un nouveau espace de noms réseau ? À quelle option de ligne de commande correspond-il ? Même question pour l'espace de noms PID et l'espaces de noms de montage.

Espace de noms réseau :

- Drapeau : `CLONE_NEWNET`
- Option : `n`

Espace de noms PID :

- Drapeau : `CLONE_NEWPID`
- Option : `p`

Espace de noms de montage :

- Drapeau : `CLONE_NS`
- Option : `m`

```

{ "mount",      optional_argument, NULL, 'm' },
{ "uts",        optional_argument, NULL, 'u' },
{ "ipc",        optional_argument, NULL, 'i' },
{ "net",        optional_argument, NULL, 'n' },
{ "pid",        optional_argument, NULL, 'p' },
{ "user",       optional_argument, NULL, 'U' },
{ "cgroup",     optional_argument, NULL, 'C' },
{ "time",       optional_argument, NULL, 'T' },

```

Image 5 : Contient la listes des options

II. Création et manipulation d'espaces de noms pour les processus avec les utilitaires Sys-Utils

1. Tout d'abord, démarrez un nouveau terminal et tapez la commande **ps**

Utilisez le PID du processus bash affiché pour exécuter : **ls -al /proc/*PID_mentionné*/ns**

- 1.1. La liste des espaces de noms correspond-elle à ce que vous avez trouvé dans nsproxy.h et man namespaces de votre VM Linux ?

IMPORTANT : En fait, la liste exacte des espaces de noms pris en charge dans votre système est déterminée par la commande **ls** précédente. Dans les versions très récentes du noyau, en plus du lien symbolique vers l'espace de noms PID appelant (pid), on a rajouté un autre lien vers l'espace de noms PID des fils (pid_for_children). Dans les deux cas, lorsque nous créons un nouvel espace de noms PID, il sera dédié uniquement pour les processus fils (et non pour le père).

PID du processus bash :

- Etape 1 : Faire la commande ps
- Etape 2 : Regarder le PID du processus bash
- Etape 3 : Faire la commande **ls -al /proc/(PID_BASH)/ns**, ou PID_BASH est le pid du processus bash

La liste des espaces de noms est la même que dans le fichier nsproxy.h. Donc par rapport à 'man namespaces', il manque toujours celui du user.

- 1.2. Faites de même avec PID=1 (ou tout autre PID de la liste des processus) : **ls -al /proc/1/ns**.

Comparez les numéros d'inodes de proc affichés avec ceux affichés par la commande précédente (question 1.1/).

(Ils doivent tous être les mêmes.) Expliquez pourquoi les numéros sont les mêmes ?

Les numéros d'inodes sont identiques car PID courant et PID 1 partagent le même namespace.

```

root@debian:~# ls -al /proc/3688/ns
total 0
dr-x--x--x 2 root root 0 févr.  2 15:26 .
dr-xr-xr-x 9 root root 0 févr.  2 15:25 ..
lrwxrwxrwx 1 root root 0 févr.  2 15:26 cgroup -> cgroup:[4026531835]
lrwxrwxrwx 1 root root 0 févr.  2 15:26 ipc -> ipc:[4026531839]
lrwxrwxrwx 1 root root 0 févr.  2 15:26 mnt -> mnt:[4026531840]
lrwxrwxrwx 1 root root 0 févr.  2 15:26 net -> net:[4026531957]
lrwxrwxrwx 1 root root 0 févr.  2 15:26 pid -> pid:[4026531836]
lrwxrwxrwx 1 root root 0 févr.  2 15:26 user -> user:[4026531837]
lrwxrwxrwx 1 root root 0 févr.  2 15:26 uts -> uts:[4026531838]
root@debian:~# man namespaces
root@debian:~# ls -al /proc/1/ns
total 0
dr-x--x--x 2 root root 0 févr.  2 13:50 .
dr-xr-xr-x 9 root root 0 févr.  2 13:41 ..
lrwxrwxrwx 1 root root 0 févr.  2 13:50 cgroup -> cgroup:[4026531835]
lrwxrwxrwx 1 root root 0 févr.  2 13:50 ipc -> ipc:[4026531839]
lrwxrwxrwx 1 root root 0 févr.  2 13:50 mnt -> mnt:[4026531840]
lrwxrwxrwx 1 root root 0 févr.  2 13:50 net -> net:[4026531957]
lrwxrwxrwx 1 root root 0 févr.  2 13:50 pid -> pid:[4026531836]
lrwxrwxrwx 1 root root 0 févr.  2 13:50 user -> user:[4026531837]
lrwxrwxrwx 1 root root 0 févr.  2 13:50 uts -> uts:[4026531838]

```

Image 6 : Utilisation des commandes pour les questions 1

2. Maintenant, exécutez les commandes suivantes consécutivement dans le même terminal :

```
ps aux
```

```
unshare --pid --fork --mount-proc bash -l (la lettre 'l' et non pas le chiffre un)
```

```
xclock &
```

```
ps aux
```

(à partir d'ici, NE tapez PAS exit dans ce terminal)

Nous avons ajouté l'option -l au bash afin de pouvoir l'identifier facilement parmi d'autres processus bash.

- 2.1. Décrivez en deux ou trois phrases les résultats.

La commande **unshare --pid --fork --mount-proc bash -l** crée un nouveau namespace et pour le /proc, il ne montre que les processus de ce namespace. Avant l'exécution des commandes, le '**ps aux**' montre tous les processus du système, alors qu'après l'exécution de la commande, le '**ps aux**' affiche seulement ceux du nouveau namespace.

```

root@debian:~# ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.0 139092  6940 ?        Ss   13:41   0:01 /sbin/init
root         2  0.0  0.0      0     0 ?        S    13:41   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        S    13:41   0:00 [ksoftirqd/0]
root         5  0.0  0.0      0     0 ?        S<   13:41   0:00 [kworker/0:0H]
root         7  0.0  0.0      0     0 ?        S    13:41   0:00 [rcu_sched]
root         8  0.0  0.0      0     0 ?        S    13:41   0:00 [rcu_bh]
root         9  0.0  0.0      0     0 ?        S    13:41   0:00 [migration/0]
root        10  0.0  0.0      0     0 ?        S<   13:41   0:00 [lru-add-drain]
root        11  0.0  0.0      0     0 ?        S    13:41   0:00 [watchdog/0]
root        12  0.0  0.0      0     0 ?        S    13:41   0:00 [cpuhp/0]
root        13  0.0  0.0      0     0 ?        S    13:41   0:00 [cpuhp/1]

```

Image 7 : La commande `ps aux` avant l'exécution des commandes

```

root@debian:~# unshare --pid --fork --mount-proc bash -l
root@debian:~# xclock &
[1] 17
root@debian:~# ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.3  0.0  21968  5756 pts/0    S    15:36   0:00 bash -l
root        17  0.2  0.0  68588  6136 pts/0    S    15:36   0:00 xclock
root        21  0.0  0.0  38304  3204 pts/0    R+   15:36   0:00 ps aux

```

Image 8 : Exécution de la commande, puis vérification du `ps aux`

2.2. Quel processus est le processus d'initialisation du nouvel espace de noms PID créé ? Quel est le nouveau PID de `xclock` ?

Le `bash -l` lancé par `unshare` est le processus d'initialisation du namespace.

Nouveau PID de `xclock` : 17

2.3. À partir d'un autre terminal, tapez :

`ps aux | grep "bash -l"`

`ps aux | grep xclock`

Exécution des commandes : `ps aux | grep "bash -l"` et `ps aux | grep xclock`

```

root@debian:~# ps aux | grep "bash -l"
root    3718  0.0  0.0   5816   664 pts/0    S    15:36   0:00 unshare --pid --fork --mount-proc bash -l
root    3719  0.0  0.0   21968  5756 pts/0    S+   15:36   0:00 bash -l
root    3763  0.0  0.0   12780   1024 pts/1    S+   15:55   0:00 grep bash -l
root@debian:~# ps aux | grep xclock
root    3735  0.0  0.0   68588   6136 pts/0    S    15:36   0:00 xclock
root    3766  0.0  0.0   12780    984 pts/1    S+   15:56   0:00 grep xclock
root@debian:~#

```

Image 9 : Exécution des commandes ci-dessus

2.3.1. Quel est le PID du processus associé à `bash -l` ?

Le PID du processus associé à `bash -l` est 3719

2.3.2. Quel est le PID du processus qui exécute l'appel système `unshare()` ? Le PID du processus qui exécute l'appel système `unshare()` est 3718

2.3.3. Quel est le PID de xclock dans l'espace de noms PID principal (espace de noms initial) ? Le PID de xclock dans l'espace de noms PID principal (espace de noms initial) est 3735

2.3.4. Comparez les numéros d'inodes de proc des trois PID précédents :

```
ls -la /proc/PID_de_bash-l/ns
```

```
ls -la /proc/PID_de_unshare/ns
```

```
ls -la /proc/PID_de_xclock/ns
```

Expliquez.

Les numéros d'inodes du PID sont différents entre unshare() et base -l/xclock, ça veut dire que unshare() a été exécuté dans un namespace différent que les deux autres. La commande unshare() va créer un nouvel espace de nom où la commande bash -l et xclock seront exécutés.

```
root@debian:~# ls -la /proc/3719/ns
total 0
dr-x--x--x 2 root root 0 févr.  2 16:04 .
dr-xr-xr-x 9 root root 0 févr.  2 15:55 ..
lrwxrwxrwx 1 root root 0 févr.  2 16:04 cgroup -> cgroup:[4026531835]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 ipc -> ipc:[4026531839]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 mnt -> mnt:[4026532234]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 net -> net:[4026531957]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 pid -> pid:[4026532235]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 user -> user:[4026531837]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 uts -> uts:[4026531838]
```

Image 10 : Inodes de base -l

```
root@debian:~# ls -la /proc/3718/ns
total 0
dr-x--x--x 2 root root 0 févr.  2 16:04 .
dr-xr-xr-x 9 root root 0 févr.  2 15:55 ..
lrwxrwxrwx 1 root root 0 févr.  2 16:04 cgroup -> cgroup:[4026531835]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 ipc -> ipc:[4026531839]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 mnt -> mnt:[4026532234]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 net -> net:[4026531957]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 pid -> pid:[4026531836]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 user -> user:[4026531837]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 uts -> uts:[4026531838]
```

Image 11 : Inodes de unshare()

```
root@debian:~# ls -la /proc/3735/ns
total 0
dr-x--x--x 2 root root 0 févr.  2 16:04 .
dr-xr-xr-x 9 root root 0 févr.  2 15:36 ..
lrwxrwxrwx 1 root root 0 févr.  2 16:04 cgroup -> cgroup:[4026531835]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 ipc -> ipc:[4026531839]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 mnt -> mnt:[4026532234]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 net -> net:[4026531957]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 pid -> pid:[4026532235]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 user -> user:[4026531837]
lrwxrwxrwx 1 root root 0 févr.  2 16:04 uts -> uts:[4026531838]
```

Image 12 : Inodes de xclock

2.4. A partir d'un autre terminal différent des deux précédents, utilisez le PID adéquat parmi les trois précédents afin d'accéder au nouvel espace de noms PID et voir les mêmes processus vus par bash -l :

```
nsenter -p -m -t PID_cible
```



```
ps aux
```

```
ls /proc
```

Les PID affichés de tous les processus doivent être les mêmes que dans le premier terminal où vous exécutez unshare, sinon quelque chose erronée s'est produit.

Exécution de :

- nsenter -p -m -t PID_cible
- ps aux
- ls /proc

2.5. Pourquoi l'option -m de nsenter est-elle requise dans la commande précédente ?

Remarque importante : Il est également possible d'accéder aux nouveaux espaces de noms à l'aide des liens symboliques des chemins /proc/.../ns :

```
nsenter --pid=/proc/PID_cible/ns/pid  
--mount=/proc/PID_cible/ns/mnt
```

L'option -m permet au shell d'afficher le dossier /proc du conteneur (où se trouve le processus). Si nous n'avions pas eu de -m, on aurait eu tous les processus du dossier /proc de la machine hôte.

2.6. A noter que la commande **lsns** permet d'afficher plus simplement la liste des espaces de noms ainsi que plusieurs informations les concernant. Testez :

```
lsns
```

```
lsns -t net
```

```
lsns -t pid
```

```
lsns -p PID_cible
```

```
lsns -o PATH,PID,PPID
```

La commande **lsns** permet d'afficher plus simplement la liste des espaces de noms ainsi que plusieurs informations les concernant.

```
root@debian:/# lsns
  NS TYPE  NPROCS  PID  USER  COMMAND
4026531835 cgroup    4    1  root  bash -l
4026531837 user     4    1  root  bash -l
4026531838 uts      4    1  root  bash -l
4026531839 ipc      4    1  root  bash -l
4026531957 net       4    1  root  bash -l
4026532234 mnt       4    1  root  bash -l
4026532235 pid       4    1  root  bash -l
root@debian:/# lsns -t net
  NS TYPE  NPROCS  PID  USER  COMMAND
4026531957 net     4    1  root  bash -l
root@debian:/# lsns -t pid
  NS TYPE  NPROCS  PID  USER  COMMAND
4026532235 pid     4    1  root  bash -l
root@debian:/# lsns -p 3719
```

Image 13 : Exécution des commandes lsns

```

root@debian:/# lsns -o PATH,PID,PPID
PATH          PID  PPID
/proc/1/ns/cgroup    1    0
/proc/1/ns/user      1    0
/proc/1/ns/uts       1    0
/proc/1/ns/ipc       1    0
/proc/1/ns/net       1    0
/proc/1/ns/mnt       1    0
/proc/1/ns/pid       1    0

```

Image 14: Exécution de la commande `lsns -o Path,PID,PPID`

3. À l'intérieur de la commande `nsenter -p -m -t PID_cible`, ou `unshare --pid --fork --mount-proc bash -l`, autrement dit à l'intérieur du nouvel espace de noms PID, exécutez : `ip a`
 Vous devriez voir la liste complète des interfaces du système. Quittez les deux bashes (`unshare` et `nsenter`). Ensuite, faites ce qui est nécessaire pour démarrer `bash -l` avec un nouvel espace de noms réseau en plus des espaces de noms PID et de montage (voir vos réponses à la question I/ 5.2/).
 Pour vérifier vos résultats : À partir de vos shell bashes `unshare` ou `nsenter`, lorsque vous tapez `ip a` vous ne devriez voir que l'interface de rebouclage (loopback `lo`) à l'état inactif `DOWN`. Ensuite, lorsque dans un `bash`, vous la faites passer à l'état actif `UP` en utilisant : `ip link set dev lo up` vous trouverez ce même statut modifié dans l'autre `bash`. Dans le cas contraire, la création ou le test de votre espace de noms a été mal réalisé.

4. Quittez tous les espaces de noms précédents et fermez tous les terminaux. Ensuite, exécutez les commandes suivantes. Pour chaque commande, fournissez une ou quelques phrases pour la décrire.
 (Notez que `root@debian :~#` est le prompt, à ne pas taper avec les commandes !)

```

root@debian :~# mkdir /mesconteneurs
root@debian :~# mkdir /mesconteneurs/pc1
root@debian :~# mount --bind --make-private
/mesconteneurs/pc1 /mesconteneurs/pc1
root@debian :~# touch /mesconteneurs/pc1/mnt
/mesconteneurs/pc1/net /mesconteneurs/pc1/pid
/mesconteneurs/pc1/uts
root@debian :~# unshare --fork --mount-proc
--mount=/mesconteneurs/pc1/mnt
--net=/mesconteneurs/pc1/net

```

```
--uts=/mesconteneurs/pc1/uts --pid sleep infinity 1  
&  
root@debian :~# cpid=$(pgrep -fn "sleep infinity  
1")
```

```
root@debian :~# mount --bind /proc/$cpid/ns/pid  
/mesconteneurs/pc1/pid
```

Remarque importante : Même si dans votre système, ns/pid_for_children est présent, les commandes précédentes fonctionnent toujours !

Afin d'entrer dans votre conteneur ou d'exécuter une commande (bash ou autre), vous tapez simplement la commande suivante (à "l'extérieur" des espaces de noms) :

```
nsenter --net=/mesconteneurs/pc1/net  
--mount=/mesconteneurs/pc1/mnt  
--pid=/mesconteneurs/pc1/pid  
--uts=/mesconteneurs/pc1/uts commande
```

Par exemple, pour changer le nom d'hôte, et donc l'invite de commandes (prompt) à l'intérieur du conteneur (bash) :

```
nsenter --net=/mesconteneurs/pc1/net  
--mount=/mesconteneurs/pc1/mnt  
--pid=/mesconteneurs/pc1/pid  
--uts=/mesconteneurs/pc1/uts hostname pc1  
ou tout simplement nsenter --uts=/mesconteneurs/pc1/uts  
hostname pc1
```

Puisque pour changer le nom d'hôte, nous n'avons besoin que de l'espace de noms UTS. De même, si nous exécutons une commande réseau, nous n'avons besoin que de l'espace de noms réseau. Par exemple

```
nsenter --net=/mesconteneurs/pc1/net ip link set  
dev lo up
```

Puis, à partir d'un autre terminal :

```
root@debian :~# nsenter --net=/mesconteneurs/pc1/net  
--mount=/mesconteneurs/pc1/mnt  
--pid=/mesconteneurs/pc1/pid  
--uts=/mesconteneurs/pc1/uts bash
```

Explications de chaque commande :

- mkdir /mesconteneurs :
Crée le répertoire à la racine du système à cause du /. Si on n'avait pas de /, on aurait créé le répertoire dans le répertoire courant.

- `mkdir /mesconteneurs/pc1` : Crée le répertoire pc1 à l'intérieur du répertoire 'mesconteneurs'.
- `mount --bind --make-private /mesconteneurs/pc1 /mesconteneurs/pc1` :
On crée un point de montage sur le répertoire lui-même avec la commande mount. --bind permet de pouvoir ajouter des options de montage comme --make-private. L'option --make-private fait que tous les montages qu'on fera à l'intérieur de ce dossier ne seront visibles que dans ce namespace, les autres (hôtes, namespaces) ne le verront pas.
- `touch /mesconteneurs/pc1/mnt /mesconteneurs/pc1/net /mesconteneurs/pc1/pid /mesconteneurs/pc1/uts` :
Crée quatre fichiers dans le dossier mesconteneurs/pc1
- `unshare --fork --mount-proc --mount=/mesconteneurs/pc1/mnt --net=/mesconteneurs/pc1/net --uts=/mesconteneurs/pc1/uts --pid sleep infinity 1 &` :
On crée un processus fils qui aura une copie des namespaces du parent pour mount, net et uts. Le --pid sleep infinity 1 garde le processus fils actif afin que tous les namespaces créés restent accessibles. Sans le sleep, le processus fils termine et tous les namespaces créés disparaissent.
- `cpid=$(pgrep -fn "sleep infinity 1")` :
Récupère le pid du processus qui a utilisé la commande "sleep infinity 1" (Ici ça sera notre processus fils)
- `mount --bind /proc/$cpid/ns/pid /mesconteneurs/pc1/pid` :
On fait un point de montage entre le namespace du processeur fils et le fichier pid dans /mesconteneurs/pc1.

4.1. Créez un autre conteneur composé de 4 espaces de noms nommé pc2 à l'aide de la même méthode employée ci-dessus (remplacez tous les 1 par 2 y compris le 1 de sleep infinity).

```

root@debian:~# cat /root/fichier1test
Texte original debian
root@debian:~# nsenter --mount=/proc/1/ns/mnt cat /root/fichier1test
Texte original debian
root@debian:~# nsenter --mount=/mesconteneurs/pc1/mnt cat /root/fichier1test
Ce fichier dans pc1
root@debian:~# nsenter --mount=/mesconteneurs/pc2/mnt cat /root/fichier1test
Ce fichier dans pc2
root@debian:~# █

```

```

root@pc2:~# hostname
pc2
root@pc2:~# ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.0   5828   668 pts/0    S+   17:30   0:00 sleep infinity
root         4  0.0  0.0  19884  3728 pts/2    S   17:33   0:00 bash
root         6  0.0  0.0  38304  3204 pts/2    R+   17:33   0:00 ps aux
root@pc2:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever

```

```

root@pc1:~# ls -lR /root/d1
/root/d1:
total 12
drwxr-xr-x 2 root root 4096 févr.  2 17:52 d11
drwxr-xr-x 2 root root 4096 févr.  2 17:52 d12
drwxr-xr-x 3 root root 4096 févr.  2 17:52 d13

/root/d1/d11:
total 0

/root/d1/d12:
total 0

/root/d1/d13:
total 4
drwxr-xr-x 2 root root 4096 févr.  2 17:53 d131

/root/d1/d13/d131:
total 4
-rw-r--r-- 1 root root 21 févr.  2 17:53 fichier1test131
root@pc1:~# cat /root/d1/d13/d131/fichier1test131
Un fichier dans d131

```

Nous avons fait un montage depuis la namespace pc1, ce qui fait qu'on voit tout le contenu du répertoire de /mesconteneurs/pc1/d1 dans /root/d1. Cela fait que depuis l'extérieur du namespace, les autres hôtes ne peuvent pas voir.

Pour l'instant, les deux conteneurs et les principaux espaces de noms du système, tous partagent le même système de fichiers, ce qui signifie qu'ils ont la même vue des points de montage et donc le même contenu de tous les répertoires et fichiers. En utilisant les espaces de noms de montage, nous pouvons changer cela. Voici un exemple simple mais concret :

En dehors des conteneurs :

```
echo "Texte original debian" > /root/fichierdest  
echo "Ce fichier dans pc1" >  
/mesconteneurs/pc1/fichierdest  
echo "Un autre dans pc2" >  
/mesconteneurs/pc2/fichierdest
```

À l'intérieur du conteneur pc1 :

```
mount --bind /mesconteneurs/pc1/fichierdest  
/root/fichierdest
```

À l'intérieur du conteneur pc2 :

```
mount --bind /mesconteneurs/pc2/fichierdest  
/root/fichierdest
```

Depuis un autre terminal, vérifiez le contenu de /root/fichierdest à l'intérieur de PC1, PC2 et à l'extérieur :

```
cat /root/fichierdest  
nsenter --mount=/proc/1/ns/mnt cat  
/root/fichierdest  
nsenter --mount=/mesconteneurs/pc1/mnt cat  
/root/fichierdest  
nsenter --mount=/mesconteneurs/pc2/mnt cat  
/root/fichierdest
```

Cela fonctionne aussi pour une arborescence de répertoires :

```
mkdir /root/d1  
mkdir /mesconteneurs/pc1/d1  
mkdir /mesconteneurs/pc1/d1/d11  
mkdir /mesconteneurs/pc1/d1/d12  
mkdir /mesconteneurs/pc1/d1/d13  
mkdir /mesconteneurs/pc1/d1/d13/d131  
echo "Un fichier dans d131" >  
/mesconteneurs/pc1/d1/d13/d131/fichierdest131
```

```
nsenter --mount=/mesconteneurs/pc1/mnt mount --bind  
/mesconteneurs/pc1/d1 /root/d1
```

Puis à l'extérieur de pc1 :

```
ls -lR /root/d1  
cat /root/d1/d13/d131/fichier131 (n'existe pas)
```

Entrez ensuite dans pc1 et tapez :

```
ls -lR /root/d1  
cat /root/d1/d13/d131/fichier131 (décrivez le  
résultat)
```

Refaites le même test avec cette fois-ci pc2 et vérifiez l'arborescence avec celle de pc1

4.2. Ajout d'interfaces virtuelles aux conteneurs (plus précisément à l'espace de noms réseau du conteneur) :

Il est possible d'utiliser le package iproute2 et son outil **ip link** pour "déplacer" les interfaces virtuelles vers les conteneurs. Iproute2 est un ensemble d'utilitaires permettant de configurer et de manipuler des réseaux de la famille TCP/IP (couches 2 et 3), y compris des fonctionnalités avancées pour le contrôle de trafic et la QoS. L'outil le plus important et le plus utilisé est la commande **ip** qui contrôle diverses configurations IPv4 et IPv6.

Pour créer un lien virtuel avec deux interfaces Ethernet virtuelles, nous exécutons la commande suivante (à l'extérieur des conteneurs) :

```
ip link add eth1 type veth peer name seth1
```

Remarque : Le lien virtuel fonctionne également comme un câble croisé.

Maintenant, pour rendre l'espace de noms réseau connu ou visible pour la commande **ip link**, exécutez : **ln -s /mesconteneurs/pc1/net /run/netns/pc1**

La commande **ip link** utilise le chemin d'accès **/run/netns** pour rechercher et identifier les espaces de noms réseau. Si ce répertoire

n'existe pas, alors à vous de le créer. Maintenant, pour associer une interface à un espace de noms réseau, exécutez la commande suivante :

```
ip link set eth1 netns pc1
```

Vérifiez que l'interface est maintenant "à l'intérieur" de pc1 :

```
Terminal - root@pc1:/
Fichier Édition Affichage Terminal Onglets Aide
root@debian:~# nsenter --net=/mycontainers/pc1/net --mount=/mycontainers/pc1/mnt --pid=/mycontainers/pc1/pid --uts=/mycontainers/pc1/uts bash
root@pc1:/# ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
10: eth1@if9: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
    link/ether 42:46:93:28:a8:31 brd ff:ff:ff:ff:ff:ff link-netnsid 0
root@pc1:/#
```

Configurez l'interface à l'intérieur de pc1 à l'aide de la commande `ip link` et `ip addr`. Utilisez /24 pour le masque de réseau (netmask)

Transférez l'autre interface `seth1` à `pc2`, configurez-la afin de pouvoir connecter les deux conteneurs `pc1` et `pc2`.



Testez avec ping et wireshark depuis l'un des conteneurs.

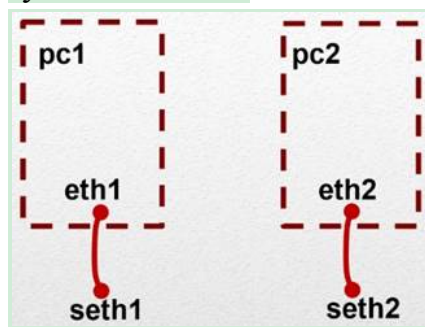
Il faut regarder dans la commande 'ip a' si les interfaces de sorties sont bien présentes sur pc1 et pc2. Ensuite, pour wireshark, il faudra qu'on associe à chaque interface une adresse ip pour pouvoir écouter sur le lien. Même si on met adresse ip à une interface, il faut faire le mapping sur linux parce que c'est pas automatique alors que sur la VM FreeBSD c'est automatique. Pour faire le mapping, il faut taper `ip link set dev <interface> up`

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	fe80::5880:aff:fe5d...	ff02::2	ICMPv6	70	Router Solicitation from 5a:80:0a:5d:c5:33
2	3.122016441	ea:43:f0:19:2f:a2	Broadcast	ARP	42	Who has 128.0.0.2? Tell 128.0.0.1
3	3.122026737	5a:80:0a:5d:c5:33	ea:43:f0:19:2f:a2	ARP	42	128.0.0.2 is at 5a:80:0a:5d:c5:33
4	3.122029603	128.0.0.1	128.0.0.2	ICMP	98	Echo (ping) request id=0x0118, seq=0x00000001
5	3.122042237	128.0.0.2	128.0.0.1	ICMP	98	Echo (ping) reply id=0x0118, seq=0x00000001
6	4.130452058	128.0.0.1	128.0.0.2	ICMP	98	Echo (ping) request id=0x0118, seq=0x00000002
7	4.130526170	128.0.0.2	128.0.0.1	ICMP	98	Echo (ping) reply id=0x0118, seq=0x00000002
8	5.153444746	128.0.0.1	128.0.0.2	ICMP	98	Echo (ping) request id=0x0118, seq=0x00000003
9	5.153457497	128.0.0.2	128.0.0.1	ICMP	98	Echo (ping) reply id=0x0118, seq=0x00000003
10	6.175968173	128.0.0.1	128.0.0.2	ICMP	98	Echo (ping) request id=0x0118, seq=0x00000004
11	6.175981223	128.0.0.2	128.0.0.1	ICMP	98	Echo (ping) reply id=0x0118, seq=0x00000004
12	7.202268414	128.0.0.1	128.0.0.2	ICMP	98	Echo (ping) request id=0x0118, seq=0x00000005
13	7.202281538	128.0.0.2	128.0.0.1	ICMP	98	Echo (ping) reply id=0x0118, seq=0x00000005

▶ Frame 1: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface 0
 ▶ Ethernet II, Src: 5a:80:0a:5d:c5:33 (5a:80:0a:5d:c5:33), Dst: IPv6mcast_02 (33:33:00:00:00:02)
 ▶ Internet Protocol Version 6, Src: fe80::5880:aff:fe5d:c533, Dst: ff02::2
 ▶ Internet Control Message Protocol v6

Image : Capture de la commande ping sur le lien depuis wireshark

4.3. Trouvez la commande qui permet de remettre à nouveau l'interface `seth1` dans le système de base. Ensuite, créez un lien virtuel connectant deux interfaces Ethernet virtuelles nommées respectivement `eth2`--`seth2`. Déplacez `eth2` vers `pc2` et configurez uniquement `eth2` (même adresse réseau). Laissez `seth2` dans le système de base.



Pour pouvoir déplacer une interface dans un autre namespace, il faut utiliser `ip link set <interface> netns <namespace>`, du coup on utilisera `ip link set seth1 netns 1` pour déplacer l'interface `seth1` dans l'hôte.

```

root@pc2:~# ip a
1: lo: <LOOPBACK,UP,LOWER UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
7: eth2@if6: <BROADCAST,MULTICAST,UP,LOWER UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 5e:df:62:ae:7f:aa brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet 128.0.2.1/24 scope global eth2
        valid_lft forever preferred_lft forever
    inet6 fe80::5cdf:62ff:feae:7faa/64 scope link
        valid_lft forever preferred_lft forever
root@pc2:~# ping 128.0.2.2
PING 128.0.2.2 (128.0.2.2) 56(84) bytes of data.
64 bytes from 128.0.2.2: icmp_seq=1 ttl=64 time=0.113 ms
64 bytes from 128.0.2.2: icmp_seq=2 ttl=64 time=0.031 ms
64 bytes from 128.0.2.2: icmp_seq=3 ttl=64 time=0.032 ms
^C
--- 128.0.2.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2106ms
rtt min/avg/max/mdev = 0.031/0.058/0.113/0.039 ms
  
```

```

root@pc1:~# ip a
1: lo: <LOOPBACK,UP,LOWER UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
5: eth1@if4: <BROADCAST,MULTICAST,UP,LOWER UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether ea:43:f0:19:2f:a2 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 128.0.0.1/24 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::e843:f0ff:fe19:2fa2/64 scope link
        valid_lft forever preferred_lft forever
root@pc1:~# ping 128.0.0.2
PING 128.0.0.2 (128.0.0.2) 56(84) bytes of data.
64 bytes from 128.0.0.2: icmp_seq=1 ttl=64 time=0.113 ms
64 bytes from 128.0.0.2: icmp_seq=2 ttl=64 time=0.033 ms
^C
--- 128.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1078ms
rtt min/avg/max/mdev = 0.033/0.073/0.113/0.040 ms

    valid_lft forever preferred_lft forever
4: seth1@if5: <BROADCAST,MULTICAST,UP,LOWER UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 5a:80:0a:5d:c5:33 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 128.0.0.2/24 scope global seth1
        valid_lft forever preferred_lft forever
    inet6 fe80::5880:aff:fe5d:c533/64 scope link
        valid_lft forever preferred_lft forever
6: seth2@if7: <BROADCAST,MULTICAST,UP,LOWER UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 8a:28:3c:74:e2:31 brd ff:ff:ff:ff:ff:ff link-netnsid 1
    inet 128.0.2.2/24 scope global seth2
        valid_lft forever preferred_lft forever
    inet6 fe80::8828:3c74:e231/64 scope link
        valid_lft forever preferred_lft forever

    valid_lft forever preferred_lft forever
root@debian:~# ping 128.0.0.1
PING 128.0.0.1 (128.0.0.1) 56(84) bytes of data.
64 bytes from 128.0.0.1: icmp_seq=1 ttl=64 time=0.125 ms
64 bytes from 128.0.0.1: icmp_seq=2 ttl=64 time=0.071 ms
64 bytes from 128.0.0.1: icmp_seq=3 ttl=64 time=0.030 ms
^C
--- 128.0.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2081ms
rtt min/avg/max/mdev = 0.030/0.075/0.125/0.039 ms
root@debian:~# ping 128.0.2.1
PING 128.0.2.1 (128.0.2.1) 56(84) bytes of data.
64 bytes from 128.0.2.1: icmp_seq=1 ttl=64 time=0.090 ms
64 bytes from 128.0.2.1: icmp_seq=2 ttl=64 time=0.038 ms
^C
--- 128.0.2.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1131ms
rtt min/avg/max/mdev = 0.038/0.064/0.090/0.026 ms

```

III. Espaces de noms réseaux et l'utilitaire ip netns du package Iproute2

La commande `ip netns` a été ajoutée pour activer et faciliter la gestion des espaces de noms réseaux : Création, suppression, configuration, surveillance, etc.

1. En utilisant la commande `ip netns`, créez deux espaces de noms réseaux nommés `ns3` et `ns4`.
Pour créer des espaces de noms, il faut utiliser la commande : `ip netns add <nom du namespace>`.
 - `ip netns add ns3`
 - `ip netns add ns4`
2. Quelle est la liste des espaces de noms maintenant (`ip netns list`) ? En plus de `ns3` et `ns4`, que représente les autres lignes affichées par `ip netns` ?
La liste des espaces de noms (`ip netns list`) contient `ns3`, `ns4`, `pc1` et `pc2`. A part `ns3` et `ns4` le reste des lignes représente les autres namespaces.

```

root@debian:~# ip netns list
ns4
ns3
pc2 (id: 1)
pc1 (id: 0)

```

3. Lancez un bash “dans” ns3, et faites de même pour ns4. Cela signifie qu’on exécute le processus bash avec le nouvel espace de noms réseau créé au lieu de l’espace de noms réseau par défaut du système. Modifiez le contenu du fichier /etc/resolv.conf à partir de ns3. La modification apparaît-elle dans ns4 ? Effectuez les configurations requises pour rendre les deux fichiers différents à l’aide du répertoire de travail de l’utilitaire ip netns.

Pour exécuter le bash dans un namespace, il faut faire : `ip netns exec ns3 bash`. Après avoir modifier le fichier /etc/resolv.conf à partir de ns3 on peut voir que le changement se fait également dans ns4. Pour rendre les deux fichiers différents, on doit :

- Pour chaque répertoire : `mkdir -p /etc/netns/ns3`
`mkdir -p /etc/netns/ns4`
- `echo "je suis dans ns3" > /etc/netns/ns3/resolv.conf`
- `echo "je suis dans ns4" > /etc/netns/ns4/resolv.conf`

On peut observer que les modifications ne sont plus appliquées aux autres namespaces.

4. Quittez tous les bashes sur ns3 et ns4. Ensuite exécutez

```

ip netns exec ns3 xclock &
ip netns exec ns3 xterm &
ip netns exec ns4 xeyes &
ip netns exec ns4 xedit &

```

Ensuite trouvez la liste des PIDs des processus qui s’exécutent sur ns3 et ns4 avec l’option pids de la commande ip netns.

```

ip netns pids ns3
ip netns pids ns4

```

Est-ce que les numéros affichés correspondent bien aux processus que vous avez lancés dans les deux espaces de noms ? Validez votre réponse avec une capture d’écran et éventuellement une ou deux phrases.

Les numéros affichés correspondent bien aux processus lancés dans ns3 et ns4.

```

root@debian:~# ip netns pids ns3
6865
6866
6868
root@debian:~# ip netns pids ns4
6869
6870

```

Image : liste des pids des processus qui s'exécutent sur ns3 et ns4

```
root@debian:~# ip netns exec ns3 xclock &
[1] 6865
root@debian:~# ip netns exec ns3 xterm &
[2] 6866
root@debian:~# ip netns exec ns4 xeyes &
[3] 6869
root@debian:~# ip netns exec ns4 xedit &
[4] 6870
```

Image : PID des processus exécutés sur les 2 namespaces

4.1. Utilisez le dernier numéro affiché par `ip netns pids ns3` pour exécuter `ip netns identify numéro_affiché` ensuite exécutez

`ip netns identify 1` (le chiffre un)

`ip netns identify 1112229`

Commentez rapidement les résultats.

Le dernier numéro affiché par `ip netns pids ns3` est 6868

- `ip netns identify 6868` : retourne bien « ns3 » car il appartient à ns3
- `ip netns identify 1` : ne retourne pas de namespace car le PID 1 appartient au namespace par défaut du système.
- `ip netns identify 1112229` : renvoie une erreur parce qu'il n'existe pas.

```
root@debian:~# ip netns identify 6868
ns3
root@debian:~# ip netns identify 1
root@debian:~# ip netns identify 1112229
Cannot open network namespace: No such file or directory
```

4.2. Tapez la commande `ip netns monitor`

Ensuite à partir d'un autre terminal, tapez :

`ip netns add nouveau`

`ip netns add unautre`

`ip netns del unautre`

`ip netns del nouveau`

En déduire le rôle de la commande `ip netns monitor :-)`

La commande `ip netns monitor` permet de voir en direct quand de nouveaux namespaces sont créés ou supprimés, lorsqu'on tape `ip netns add` ou `ip netns del`, on verra depuis le terminal qui a utilisé la commande `monitor`, `add <nom de la namespace>` ou `del <nom de la namespace>`.

```
root@debian:~# ip netns monitor
add nouveau
add unautre
delete unautre
delete nouveau
```

- 4.3. Enfin, l'option `list-id` permet d'afficher les espaces de noms réseaux avec un identifiant (`netnsid`), et l'option `set` permet d'attribuer un identifiant à un espace de noms réseau sans identifiant. Les identifiants sont rajoutés par les utilitaires en cas de besoin afin de retrouver et identifier numériquement et facilement les espaces de noms réseaux. Testez-les rapidement :

```
ip netns set ns3 33
```

```
ip netns set ns4 44
```

```
ip netns list-id
```

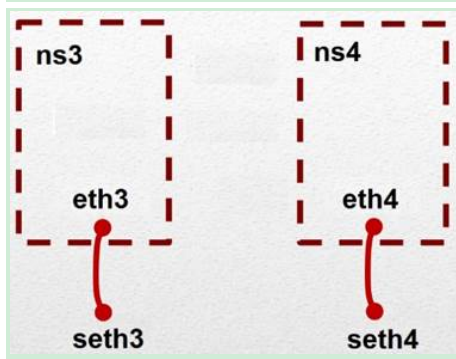
L'option `list-id` permet d'afficher les espaces de noms réseaux avec un identifiant (`netnsid`), et l'option `set` permet d'attribuer un identifiant à un espace de noms réseau sans identifiant.

```
root@debian:~# ip netns set ns3 33
root@debian:~# ip netns set ns4 44
root@debian:~# ip netns list-id
nsid 0 (iproute2 netns name: pc1)
nsid 1 (iproute2 netns name: pc2)
nsid 33 (iproute2 netns name: ns3)
nsid 44 (iproute2 netns name: ns4)
```

Remarque : Les options de l'utilitaire `ip netns` sont particulièrement utiles pour automatiser les manipulations des espaces de noms réseaux et les contrôler facilement.

5. Créer deux liens virtuels connectant deux interfaces Ethernet virtuelles nommées respectivement `eth3--seth3` et `eth4--seth4`. On n'a pas besoin de la commande `ln -s ... /run/netns/...` puisque `/run/netns` est déjà utilisé par `ip netns`. Ensuite, associez respectivement les interfaces `eth3` et `eth4` aux espaces de noms réseaux `ns3` et `ns4`. Vérifiez et configurez les interfaces à l'intérieur de `ns3` et `ns4`. Utilisez la même adresse réseau que précédemment.

Vous connecterez les autres interfaces seth3 et seth4 plus tard.



Commandes saisies :

- ip link add eth3 type veth peer name seth3
- ip link add eth4 type veth peer name seth4
- ip link set eth3 netns ns3
- ip link set eth4 netns ns4
- ip netns exec ns3 ip addr add 128.0.0.3/24 dev eth3
- ip netns exec ns4 ip addr add 128.0.0.4/24 dev eth4

Avec les commandes pings, on a vérifié que la connectivité entre les ns3 et ns4 fonctionne.

6. Créez une interface de type bridge nommée monbr, comme celle qu'on a créée avec Virtual Manager, mais avec la commande ip link. Ensuite, rattachez les interfaces seth1, seth2, seth3 et seth4 aux bridges et testez la connectivité entre les 4 espaces de noms réseaux pc1, pc2, ns3 et ns4.

N'oubliez pas d'activer (« UPper ») toutes les interfaces

Commandes saisies :

- ip link add monbr type bridge
- ip link set monbr up
- ip link set seth1 up
- ip link set seth2 up
- ip link set seth3 up
- ip link set seth4 up
- ip link set seth1 master monbr
- ip link set seth2 master monbr
- ip link set seth3 master monbr
- ip link set seth4 master monbr
- ip netns exec pc1 ping 128.0.0.2 (Marche)
- ip netns exec pc1 ping 128.0.0.3 (Marche)
- ip netns exec pc1 ping 128.0.0.4 (Marche)

```

root@debian:~# ip netns exec pc1 ping -c 2 128.0.0.3
PING 128.0.0.3 (128.0.0.3) 56(84) bytes of data.
64 bytes from 128.0.0.3: icmp_seq=1 ttl=64 time=0.068 ms
64 bytes from 128.0.0.3: icmp_seq=2 ttl=64 time=0.086 ms

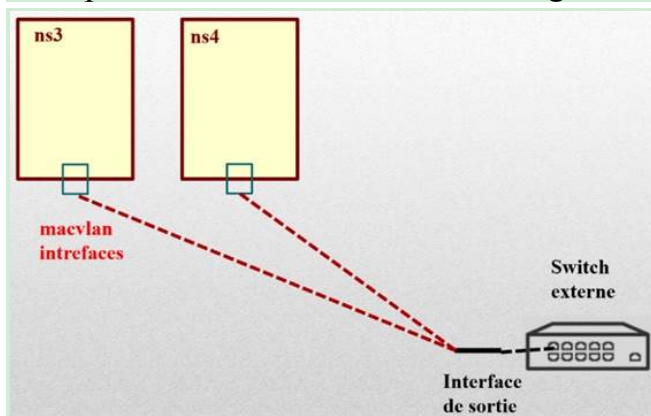
--- 128.0.0.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1024ms
rtt min/avg/max/mdev = 0.068/0.077/0.086/0.009 ms
root@debian:~# ip netns exec pc1 ping -c 2 128.0.0.4
PING 128.0.0.4 (128.0.0.4) 56(84) bytes of data.
64 bytes from 128.0.0.4: icmp_seq=1 ttl=64 time=0.060 ms
64 bytes from 128.0.0.4: icmp_seq=2 ttl=64 time=0.056 ms

--- 128.0.0.4 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1015ms
rtt min/avg/max/mdev = 0.056/0.058/0.060/0.002 ms
root@debian:~# ip netns exec pc1 ping -c 2 128.0.0.2
PING 128.0.0.2 (128.0.0.2) 56(84) bytes of data.

--- 128.0.0.2 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1039ms

```

7. A l'instar des interfaces de type macvtap, il est possible de connecter les espaces de noms réseaux, et donc leurs conteneurs, directement en utilisant les interfaces de type macvlan, sans les liens virtuels (veth) ni le bridge. L'interface macvlan fonctionne de manière similaire à l'interface macvtap avec aussi les 5 modes : Pont, VEPA, privé, passthru et source. Ce dernier mode permet de définir une liste d'adresses MAC sources autorisées à se connecter à travers l'interface macvlan. Cela permet de créer des LANs virtuels basées sur les adresses MAC sans étiquettes supplémentaires dans les entêtes des trames MAC, et sans impliquer les ports d'un bridge comme pour les VLANs classiques. Détachez l'interface eth3 et eth4 des espaces de noms ns3 et ns4 afin de déconnecter les deux espaces de noms. Ensuite, créez deux interfaces de type macvlan en mode Pont (bridge), nommées respectivement macvlan3 et macvlan4, et qui sont rattachées à l'interface de sortie de la VM Debian Linux CRV4 (enp0s3). Ensuite, déplacez les deux interfaces macvlan vers les espaces de noms ns3 et ns4. Configurez et testez la connectivité.



Remarque : Un “inconvenient” de ce type d’interface et qu’il ne permet pas la création de scénarios avancés en cas de besoin. Un seul scénario est possible : des espaces de noms réseaux (conteneurs) connectés directement à travers une interface de sortie. L’avantage est qu’il permet de transférer la responsabilité de commutation aux switch externe (en mode VEPA). Par ailleurs, il simplifie les configurations en évitant la création de liens virtuels et de bridge ou autres composants.

Commandes saisies :

- ip netns exec ns3 ip link set eth3 netns 1
- ip netns exec ns4 ip link set eth4 netns 1

Création des interfaces macvlan en mode Bridge :

- ip link add macvlan3 link enp0s3 type macvlan mode bridge
- ip link add macvlan4 link enp0s3 type macvlan mode bridge
- ip link set macvlan3 netns ns3
- ip link set macvlan4 netns ns4
- ip netns exec ns3 ip addr add 128.0.0.103/24 dev macvlan3
- ip netns exec ns3 ip link set macvlan3 up
- ip netns exec ns3 ip link set lo up
- ip netns exec ns4 ip addr add 128.0.0.104/24 dev macvlan4
- ip netns exec ns4 ip link set macvlan4 up
- ip netns exec ns4 ip link set lo up
- ip netns exec ns3 ping -c 2 128.0.0.104
- ip netns exec ns4 ping -c 2 128.0.0.103

```
root@debian:~# ip netns exec ns3 ping -c 2 128.0.0.104
PING 128.0.0.104 (128.0.0.104) 56(84) bytes of data.
64 bytes from 128.0.0.104: icmp_seq=1 ttl=64 time=0.067 ms
64 bytes from 128.0.0.104: icmp_seq=2 ttl=64 time=0.032 ms

--- 128.0.0.104 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1059ms
rtt min/avg/max/mdev = 0.032/0.049/0.067/0.018 ms
root@debian:~# ip netns exec ns4 ping -c 2 128.0.0.103
PING 128.0.0.103 (128.0.0.103) 56(84) bytes of data.
64 bytes from 128.0.0.103: icmp_seq=1 ttl=64 time=0.088 ms
64 bytes from 128.0.0.103: icmp_seq=2 ttl=64 time=0.073 ms

--- 128.0.0.103 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1049ms
rtt min/avg/max/mdev = 0.073/0.080/0.088/0.011 ms
```

IV. Contrôle de ressources des processus de conteneurs : Groupes de contrôle - cgroups

Les logiciels de conteneurisation peuvent également utiliser la fonctionnalité cgroups du noyau Linux pour contrôler, limiter et surveiller différemment les ressources du système (mémoire, CPU, etc.) fournis aux processus de conteneurs. L’interface cgroup du noyau est fournie par un pseudo-système de fichiers appelé

cgroupfs (/sys/fs/cgroup). Le suivi et la limitation des ressources sont implémentés dans un ensemble de sous-systèmes (contrôleurs) par type de ressource.

Un cgroup est un groupe de processus liés à un ensemble de limites ou de paramètres définis via le système de fichiers cgroup. Les cgroups sont organisés hiérarchiquement via la création, la suppression et le renommage de sous-répertoires dans le système de fichiers cgroup. Testons un exemple concret.

1. Installez d'abord l'utilitaire **stress** qui sera utilisé pour allouer facilement de la mémoire à des fins de test : **apt install stress**

À partir d'un autre terminal, exécutez le programme **stress** comme suit :

```
stress -v --vm-bytes 13M --vm-keep --vm 1
```

Ensuite, observez l'état de la mémoire du processus de **stress** à l'aide de son PID affiché dans les dernières lignes imprimées par le programme **stress** :

```
cat /proc/stressPID/status
```

```
VmStk:      132 kB
VmExe:       20 kB
VmLib:      2804 kB
VmPTE:       60 kB
VmPMD:       12 kB
VmSwap:       0 kB
HugetlbPages: 0 kB
Threads:      1
SigQ:    0/128624
SigPnd: 0000000000000000
ShdPnd: 0000000000000000
SigBlk: 0000000000000000
SigIgn: 0000000000000000
SigCgt: 0000000000000000
CapInh: 0000000000000000
CapPrm: 0000003fffffffff
CapEff: 0000003fffffffff
CapBnd: 0000003fffffffff
CapAmb: 0000000000000000
Seccomp: 0
Speculation_Store_Bypass: vulnerable
Cpus_allowed: 3f
Cpus_allowed list: 0-5
Mems_allowed: 00000000,00000001
Mems_allowed list: 0
voluntary cxt switches: 71
nonvoluntary cxt switches: 125404
```

2. Dans un autre terminal, créez un nouveau répertoire appelé **pc1** à l'intérieur du contrôleur de mémoire dans cgroupfs :

```
mkdir /sys/fs/cgroup/memory/pc1
```

```
ls /sys/fs/cgroup/memory/pc1
```

Limiter la mémoire allouée aux processus appartenant à ce groupe en modifiant la valeur du fichier **memory.limit_in_bytes** qui se trouve dans le nouveau répertoire **PC1** :

```
cd /sys/fs/cgroup/memory/pc1
```

```
echo $((10 * 1024 * 1024)) > memory.limit_in_bytes
```

On définit la limite d'un processus peut avoir. On initialise à 10 Mo.

```
root@debian:~# mkdir /sys/fs/cgroup/memory/pc1
root@debian:~# ls /sys/fs/cgroup/memory/pc1
cgroup.clone_children      memory.limit_in_bytes
cgroup.event_control       memory.max_usage_in_bytes
cgroup.procs               memory.move_charge_at_immigrate
memory.failcnt             memory.numa_stat
memory.force_empty         memory.oom_control
memory.kmem.failcnt        memory.pressure_level
memory.kmem.limit_in_bytes memory.soft_limit_in_bytes
memory.kmem.max_usage_in_bytes memory.stat
memory.kmem.slabinfo       memory.swappiness
memory.kmem.tcp.failcnt    memory.usage_in_bytes
memory.kmem.tcp.limit_in_bytes memory.use_hierarchy
memory.kmem.tcp.max_usage_in_bytes notify_on_release
memory.kmem.tcp.usage_in_bytes tasks
memory.kmem.usage_in_bytes
root@debian:~# cd /sys/fs/cgroup/memory/pc1
root@debian:/sys/fs/cgroup/memory/pc1# echo $((10 * 1024 * 1024)) > memory.limit_in_bytes
root@debian:/sys/fs/cgroup/memory/pc1#
```

3. Pour inclure le processus bash, où nous allons exécuter les tests de stress, au cgroup du contrôleur de mémoire créé, il suffit d'ajouter le PID du processus bash dans le fichier cgroups.procs :

```
echo $$ # Depuis le terminal où vous allez exécuter les tests de stress (vous permet d'obtenir le PID du processus bash).
```

```
echo PID_obtenu_par_echo$$ > cgroup.procs # L'autre terminal
```

Maintenant, exécutez à nouveau le programme de stress (à partir du bash où vous avez lancé echo \$).

Observez le contenu de **cgroup.procs**, le contenu de **/proc/stressPID/status**

et le résultat de la commande **ps aux | grep "stressPID \\\ VSZ"** (Le RSS doit être toujours inférieur à 10)

Décrivez tous les résultats.

Vous pouvez arrêter et relancer le programme de stress avec d'autres valeurs de vm-bytes pour confirmer vos observations. Vous pouvez également modifier la limite de la mémoire. Cependant, si vous choisissez une valeur très faible de la limite de mémoire, le processus bash et éventuellement son terminal seront tués par le noyau.

Remarque : En utilisant cette fonctionnalité, il est possible d'associer des cgroups à tous les processus d'un conteneur afin de créer un environnement d'exécution spécifique pour l'ensemble du conteneur. Le seul problème est que les processus d'un conteneur peuvent connaître et éventuellement modifier les informations concernant les cgroups d'autres conteneurs, même les cgroups de l'hôte système. Ainsi, les cgroups ont été également virtualisés avec des espaces de noms afin de

résoudre le problème. En outre, les espaces de noms cgroups facilitent la migration et la gestion des conteneurs puisque l'isolation des processus est plus complète.

Pour tester la limite, on va utiliser la commande `stress -v --vm-bytes 13M --vm-keep --vm 1` pour allouer 13Mo au processus. On peut voir dans l'image ci-dessous que le processeur est limité à 10 Mo de RAM mais que le reste est parti dans le disque car on a activé le swap. Sinon, on n'aurait pas pu lancer le stress.

```
root@debian:/sys/fs/cgroup/memory/pcl# ps aux | grep "7677 \| VSZ"
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root      7677  7.9  0.0  20588 10216 pts/3    D+   16:21   0:04 stress -v --vm-bytes 13M
--vm-keep --vm 1
root      7681  0.0  0.0  12780   956 pts/5      S+   16:22   0:00 grep 7677 \| VSZ
```

```
root@debian:/sys/fs/cgroup/memory/pcl# ps aux | grep "7677 \| VSZ"
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root      7677  7.9  0.0  20588 10216 pts/3    D+   16:21   0:04 stress -v --vm-bytes 13M
--vm-keep --vm 1
root      7681  0.0  0.0  12780   956 pts/5      S+   16:22   0:00 grep 7677 \| VSZ
root@debian:/sys/fs/cgroup/memory/pcl# ps aux | grep "7677 \| VSZ"
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root      7687  0.0  0.0  12780   972 pts/5      S+   16:26   0:00 grep 7677 \| VSZ
root@debian:/sys/fs/cgroup/memory/pcl# ps aux | grep "7684 \| VSZ"
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root      7684 37.7  0.0   9324  2776 pts/3      R+   16:26   0:03 stress -v --vm-bytes 2M
--vm-keep --vm 1
root      7690  0.0  0.0  12780   940 pts/5      S+   16:26   0:00 grep 7684 \| VSZ
root@debian:/sys/fs/cgroup/memory/pcl# ps aux | grep "7694 \| VSZ"
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root      7694  6.1  0.0  32876 10180 pts/3    D+   16:27   0:00 stress -v --vm-bytes 25M
--vm-keep --vm 1
root      7696  0.0  0.0  12780   960 pts/5      S+   16:27   0:00 grep 7694 \| VSZ
```

V. Docker en tant que logiciel de conteneurisation basée sur les espaces de noms

À l'instar de LXC et d'autres projets de conteneurisation, docker utilise notamment les espaces de noms et les cgroups pour isoler les applications dans des conteneurs. docker est déjà installé sur votre machine virtuelle Linux. Les images ou les fichiers d'images qui sont associés au logiciel docker, contiennent un système de fichiers partiel ou complet (arborescence commençant par '/') qui sera utilisé pour exécuter les programmes (processus) à l'intérieur du conteneur. Ces processus ne verront que ces données sauf si on rattache explicitement d'autres répertoires ou périphériques à partir du système de fichiers de la machine hôte.

Commençons par un exemple simple utilisant l'image getting-started fourni par docker. L'image contient un système de fichiers minimal pour présenter un tutoriel. En particulier, un serveur web, appelé nginx, fonctionnera dans le conteneur en tant que premier processus. L'image contient également plusieurs utilitaires tels que echo, ping et ip

1. Nous souhaitons exécuter la commande suivante :

```
docker run -d -p 8088:80 --name dctest
```

```
docker/getting-started
```

Le nom de l'image est docker/getting-started. Elle sera automatiquement téléchargée à partir du dépôt Docker Hub. Ensuite, la commande démarre un conteneur à l'aide du système de fichiers de l'image. Nous l'appelons dctest. Le port hôte 8088 est mappé au port 80 du conteneur sur lequel le serveur nginx est à l'écoute.

Néanmoins, si votre système est connecté à travers un proxy http alors, il faut d'abord le mentionner au daemon docker :

```
gedit /lib/systemd/system/docker.service
```

dans la section [Service] rajoutez les deux lignes

```
Environment="HTTP_PROXY=http://proxy:3128/"
```

```
Environment="HTTPS_PROXY=http://proxy:3128/"
```

Sauvegardez. Ensuite, redémarrez le service docker comme suit :

```
systemctl daemon-reload
```

```
systemctl restart docker
```

Après avoir exécuter la commande docker run mentionnée ci-dessus, vérifiez maintenant la liste des conteneurs Docker :

```
docker container ls # ou
```

```
docker ps
```

Vous pouvez également vérifier que le serveur Web est en cours d'exécution et accessible via le port local hôte 8088 via l'URL <https://localhost:8088/> dans le navigateur Web hôte.

Pour voir l'arborescence complète du système de fichiers associés au conteneur : **docker exec dctest ls -lR /**

La liste des processus en cours d'exécution dans le conteneur :

```
docker top dctest # Deux PIDs sont affichés par cette commande (un premier et un deuxième).
```

1.1. Expliquez et comparez les résultats obtenus par ces commandes (à l'extérieur du conteneur) :

```
ls -al /proc/premierPID/ns
```

```
ls -al /proc/deuxiemePID/ns
```

```
ls -al /proc/1/ns
```

```
ps aux | grep nginx
```

```
docker exec dctest ps aux
```

On peut voir que les deux processus créés par le docker ils ont les mêmes inodes, cependant ils sont différents du processus 1 (même pour cgroup et user, diff pour mnt net pid ipc), cela implique que les

deux processus ne sont pas dans le même conteneur que le processus 1. Ensuite, avec les commandes `ps aux | grep nginx` et `docker exec dctest ps aux`, on observe que les processus des dockers sont visibles depuis système root. De plus, si nous regardons depuis le conteneur docker, on peut voir que le PID 1 est la commande master alors que dans le système root c'est PID 8714.

```
root@debian:~# ls -al /proc/8714/ns
total 0
dr-x--x--x 2 root root 0 févr.  9 16:40 .
dr-xr-xr-x 9 root root 0 févr.  9 16:40 ..
lrwxrwxrwx 1 root root 0 févr.  9 16:46 cgroup -> cgroup:[4026531835]
lrwxrwxrwx 1 root root 0 févr.  9 16:40 ipc -> ipc:[4026532781]
lrwxrwxrwx 1 root root 0 févr.  9 16:40 mnt -> mnt:[4026532779]
lrwxrwxrwx 1 root root 0 févr.  9 16:40 net -> net:[4026532784]
lrwxrwxrwx 1 root root 0 févr.  9 16:40 pid -> pid:[4026532782]
lrwxrwxrwx 1 root root 0 févr.  9 16:46 user -> user:[4026531837]
lrwxrwxrwx 1 root root 0 févr.  9 16:40 uts -> uts:[4026532780]
root@debian:~# ls -al /proc/8770/ns
total 0
dr-x--x--x 2 systemd-network systemd-journal 0 févr.  9 16:47 .
dr-xr-xr-x 9 systemd-network systemd-journal 0 févr.  9 16:41 ..
lrwxrwxrwx 1 systemd-network systemd-journal 0 févr.  9 16:47 cgroup -> cgroup:[4026531835]
lrwxrwxrwx 1 systemd-network systemd-journal 0 févr.  9 16:47 ipc -> ipc:[4026532781]
lrwxrwxrwx 1 systemd-network systemd-journal 0 févr.  9 16:47 mnt -> mnt:[4026532779]
lrwxrwxrwx 1 systemd-network systemd-journal 0 févr.  9 16:47 net -> net:[4026532784]
lrwxrwxrwx 1 systemd-network systemd-journal 0 févr.  9 16:47 pid -> pid:[4026532782]
lrwxrwxrwx 1 systemd-network systemd-journal 0 févr.  9 16:47 user -> user:[4026531837]
lrwxrwxrwx 1 systemd-network systemd-journal 0 févr.  9 16:47 uts -> uts:[4026532780]
```

```
root@debian:~# ls -al /proc/1/ns
total 0
dr-x--x--x 2 root root 0 févr.  2 13:50 .
dr-xr-xr-x 9 root root 0 févr.  2 13:41 ..
lrwxrwxrwx 1 root root 0 févr.  2 13:50 cgroup -> cgroup:[4026531835]
lrwxrwxrwx 1 root root 0 févr.  2 13:50 ipc -> ipc:[4026531839]
lrwxrwxrwx 1 root root 0 févr.  2 13:50 mnt -> mnt:[4026531840]
lrwxrwxrwx 1 root root 0 févr.  2 13:50 net -> net:[4026531957]
lrwxrwxrwx 1 root root 0 févr.  2 13:50 pid -> pid:[4026531836]
lrwxrwxrwx 1 root root 0 févr.  2 13:50 user -> user:[4026531837]
lrwxrwxrwx 1 root root 0 févr.  2 13:50 uts -> uts:[4026531838]
```

```
root@debian:~# ps aux | grep nginx
root      8714  0.0  0.0   7620  5024 ?        Ss   16:40   0:00 nginx: master process nginx -g daemon off;
systemd+  8770  0.0  0.0   8076  1956 ?        S    16:40   0:00 nginx: worker process
systemd+  8771  0.0  0.0   8076  1956 ?        S    16:40   0:00 nginx: worker process
systemd+  8772  0.0  0.0   8076  1956 ?        S    16:40   0:00 nginx: worker process
systemd+  8773  0.0  0.0   8076  1956 ?        S    16:40   0:00 nginx: worker process
systemd+  8774  0.0  0.0   8076  1956 ?        S    16:40   0:00 nginx: worker process
systemd+  8775  0.0  0.0   8076  1956 ?        S    16:40   0:00 nginx: worker process
root      9296  0.0  0.0  12780  1028 pts/3    S+   16:51   0:00 grep nginx
root@debian:~# docker exec dctest ps aux
PID    USER    TIME    COMMAND
1  root    0:00    nginx: master process nginx -g daemon off;
29  nginx   0:00    nginx: worker process
30  nginx   0:00    nginx: worker process
31  nginx   0:00    nginx: worker process
32  nginx   0:00    nginx: worker process
33  nginx   0:00    nginx: worker process
34  nginx   0:00    nginx: worker process
61  root    0:00    ps aux
```

1.2. Expliquez brièvement les résultats de ces commandes :

`docker exec dctest hostname`

`nsenter -u -t premierPID hostname dctest`

`docker exec dctest hostname`

`nsenter -u -t deuxiemePID hostname`

Note: Si vous souhaitez exécuter des commandes alors que vous êtes « à l'intérieur » du conteneur dctest, exécutez la commande suivante (bash n'est pas installé dans l'image du système de fichiers de dctest, et nous devons exiger le mode interactif et un tty) :

```
docker exec -it dctest sh # et vous obtenez le prompt suivant « à l'intérieur »  
/ #
```

Depuis le conteneur docker, on peut voir que le hostname est 3f32d31a9636. On va modifier le hostname à partir de nsenter parce que docker n'a pas la permission de pouvoir modifier le hostname. On peut voir, après, que le hostname est bien dctest. Ça veut également dire que tout ce que nous faisons dans le conteneur peut être observé ou modifié par la personne qui a le privilège root, car il a accès aux namespaces et aux processus.

```
root@debian:~# docker exec dctest hostname  
3f32d31a9636  
root@debian:~# nsenter -u -t 8714 hostname dctest  
root@debian:~# docker exec dctest hostname  
dctest  
root@debian:~# nsenter -u -t 8770 hostname  
dctest
```

1.3. A l'extérieur du conteneur, exécutez la commande **ps -eo pid,ppid,command f**

1.3.1. Quel est le processus père du serveur web ? Quel est le père de ce père ? (Normalement, il s'agit du runtime du conteneur)

1ere ligne = père du père du serveur Web (runtime)

2eme ligne = père du serveur Web

```
535      1 /usr/bin/containerd  
8699    535 \_ containerd-shim -namespace moby -workdir /var/lib/containerd/io.containerd.runtime.v1.linux/moby/3f3  
8714    8699 \_ nginx: master process nginx -g daemon off;  
8770    8714 \_ nginx: worker process  
8771    8714 \_ nginx: worker process  
8772    8714 \_ nginx: worker process  
8773    8714 \_ nginx: worker process  
8774    8714 \_ nginx: worker process  
8775    8714 \_ nginx: worker process  
569      1 /usr/sbin/sshd -D
```

1.3.2. En réalité, il y a un programme important qui ne s'affiche pas dans la liste des processus car il s'est terminé. Ce programme a été utilisé pour créer l'environnement d'exécution des processus du conteneur. Il s'agit de la commande **runc**, l'exécuteur direct du conteneur ou en anglais low-level container runtime. **runc** est conforme aux spécifications de [l'OCI \(Open Container Initiative\)](#). En particulier, c'est **runc** qui crée les espaces de noms.

D'abord vérifiez que le runtime est bien **runc** :

```
docker info | grep -i runtime
```

Trouvez le ID complet du conteneur avec la commande

```
docker ps --no-trunc | cut -f1 -d' ' # (-f un  
et non la lettre 'L')
```

Ensuite, consultez le fichier de configuration du conteneur que runc a utilisé et qui se trouve dans le répertoire /run/containerd. L'ID du conteneur devrait vous aider à trouver le chemin complet vers ce fichier. Il s'agit d'un fichier json (JavaScript Object Notation).

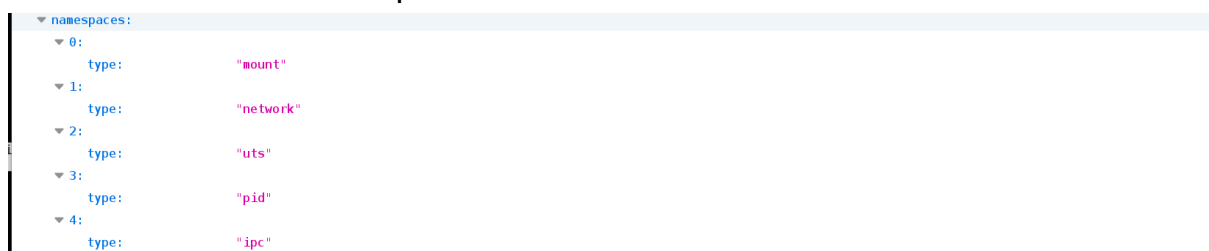
Ouvrez-le avec firefox ou un autre navigateur. Par exemple :

```
firefox /run/containerd/chemin/container_ID/config.json
```

a) Quels sont les espaces de nom créés par runc ?

Les espaces créés par runc sont :

- mount
- network
- uts
- pid
- ipc



b) Quel est l'emplacement "réel" du fichier /etc/resolv.conf du système de fichiers du conteneur ? (Donnez le chemin complet sur le système hôte)

L'emplacement réel du fichier resolv.conf est situé à /var/lib/docker/containers/3f...92c/resolv.conf



1.3.3. Trouvez le répertoire de runtime maintenu par docker (containerd-shim) :

```
ps aux | grep runtime-root #(ensuite regardez  
l'option --runtime-root)
```

Le sous répertoire du répertoire trouvé ci-dessus vous permet d'accéder et éventuellement de manipuler les conteneurs avec la commande runc. Exemple :


```
runc --root
/chemin_complet_du_sous_repertoire list
```

(Le BUNDLE correspond au répertoire où vous avez trouvé le fichier config.json)

Il y a aussi un fichier state.json sous /chemin_complet_du_sous_repertoire/containerID/ qui contient les informations du conteneur en cours d'exécution.

a) Quel répertoire a été créé dans le système de fichier cgroupfs (/sys/fs/cgroup) afin de contrôler les ressources du conteneur ? Le répertoire créé dans le système de fichier cgroupfs afin de contrôler les ressources du conteneur se trouve sur le chemin :

```
/sys/fs/cgroup/cpuset/docker/3f32d31a...a7685392c/
```

b) Avec le contrôleur cpuset, trouvez la liste des CPUs pouvant être utilisés par les processus du conteneur. Tous les processeurs peuvent être utilisés par le conteneur. (Processeur 0, processeur 1, etc..)

```
root@debian:~# cat /sys/fs/cgroup/cpuset/docker/3f32d31a96364c83269d09780b376f35fb88bdf90338b3d0d9f1be5a7685392c/cpuset.cpus
0-5
```

c) Que représente la liste des PIDs mentionnés dans le fichier cgroup.procs ? C'est la liste des processus créés par le docker vu depuis le système hôte.

```
root@debian:~# cat /sys/fs/cgroup/cpuset/docker/3f32d31a96364c83269d09780b376f35fb88bdf90338b3d0d9f1be5a7685392c/cgroup.procs
8714
8770
8771
8772
8773
8774
8775
```

2. Réseaux de conteneurs. Il existe plusieurs méthodes pour connecter les conteneurs docker (et les conteneurs en général) entre eux et à d'autres réseaux.

2.1. Commençons par la plus simple en utilisant la commande docker :

```
docker network create -d bridge
mydefaultbridge
docker run -d --net mydefaultbridge --name dc5
docker/getting-started
docker run -d --name dc6
docker/getting-started # pas de --net pour tester une autre
méthode
docker exec dc5 ifconfig
```



```
docker exec dc6 ifconfig
```

```
docker network connect mydefaultbridge dc6 #
```

L'autre méthode

```
docker exec dc6 ifconfig
```

```
docker exec dc5 ping 1_adresse_adéquate_de_dc6
```

Question : docker a-t-il utilisé la même technique que celle décrite dans la diapositive 59, page 30 du cours 2 ? Si oui, alors prouvez-le. Pour vous aider, vous pouvez exécuter et observer l'affichage de ces commandes :

```
ip a
```

```
docker network list
```

```
ip a | grep ": br-"
```

```
ip a | grep "master br"
```

Remarque : L'option **-d** permet la création d'une interface réseau supportée par le noyau et qui apparaît dans le TYPE de la commande `ip link` (`ip link help`) à condition qu'il soit implanté par docker)

On peut voir que les deux interfaces veth sont connectées au bridge, ce qui prouve bien qu'on utilise la même technique que celle décrite dans la diapositive 59, page 30 du cours 2.

```
root@debian:~# ip a | grep ": br-"
17: br-a018964ff25f: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
root@debian:~# ip a | grep "master br"
19: veth9a3b0bc@if18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-a018964ff25f state UP group default
23: veth9bcb62b@if22: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-a018964ff25f state UP group default
```

2.2. Utiliser la sous-commande **nomaster** de `ip link` pour détacher les interfaces veth de mydefaultbridge (br-something). Vous pouvez également, si vous préférez, renommer les interfaces veth en noms plus simples tels que `seth5` et `seth6`. Nous les reconnecterons plus tard avec `seth1`, `seth2`, `seth3` et `seth4`.

```
root@debian:~# ip a | grep "br-"
17: br-a018964ff25f: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-a018964ff25f
19: veth9a3b0bc@if18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-a018964ff25f state UP group default
23: veth9bcb62b@if22: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master br-a018964ff25f state UP group default
root@debian:~# bridge link show dev br-a018964ff25f
root@debian:~# ip link set veth9a3b0bc no master
Command line is not complete. Try option "help"
root@debian:~# ip link set veth9a3b0bc nomaster
root@debian:~# ip link set veth9bcb62b nomaster
root@debian:~# ip link set veth9a3b0bc nomaster
root@debian:~# ip link set veth9a3b0bc down
root@debian:~# ip link set veth9a3b0bc name seth5
root@debian:~# ip link set seth5 up
root@debian:~# ip link set veth9bcb62b down
root@debian:~# ip link set veth9bcb62b name seth6
root@debian:~# ip link set seth6 up
```

Note : Il est possible d'effectuer toutes les configurations réseau en utilisant `ip`, `ip netns` et/ou `nsenter -n`, ce qui s'avère beaucoup plus rapide que la commande `docker`. Par conséquent, il est recommandé d'adopter cette approche si le nombre de conteneurs et les configurations sont nombreuses et/ou répétitives.